

ENGLISH FOR WORK / SEPTEMBER 2026

Software Development English



Vocabulary & phrasebook

Keep precise meanings and useful expressions close. Retrieve the words, then use them in a complete message.

PRACTICE THAT TRANSFERS TO WORK

Read a realistic case. Study complete conversations. Find the right language, speak, get feedback, and try again.

Eight lessons | Intermediate to advanced | Independent or classroom study

For Software developers, application engineers, code reviewers, and maintainers.

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

All scenarios, figures, and model responses are fictional teaching material. Use invented details in practice.

[Open this course on English Ladder](#)

Make vocabulary usable

Knowing a definition is a start. To use a term well, explain it in plain English, connect it to a case, and choose a natural sentence around it.

- Read five terms, cover their definitions, and recall the meanings.
- Sort a pair you might confuse and explain the difference.
- Use one term in a sentence about a fictional case.
- Ask a partner to explain the sentence without repeating the specialist word.
- Return to the same terms on another day and test recall again.

Abbreviations need context

Expand an abbreviation on first use when the listener needs it. Some abbreviations have different meanings across professions: API can refer to an application programming interface or an active pharmaceutical ingredient. The course context determines the meaning.

Definitions have limits

These are concise language-learning explanations, not complete legal, clinical, or technical specifications. Use the applicable authoritative definition for regulated or operational decisions.

Field vocabulary A-Z

backward compatibility

The ability of a changed component to continue supporting behavior or inputs relied on by existing consumers.

benchmark

A defined measurement exercise used to compare performance under stated workloads and conditions.

build artifact

A generated software output, such as a package or executable, produced from a particular build.

deprecation

A notice that an interface or feature is discouraged for future use, often ahead of possible removal.

diff

A representation showing added, removed, or changed content between two versions.

direct dependency

A component explicitly required by the application or package being discussed.

feature flag

A control that enables, disables, or varies specified application behavior without necessarily changing the deployed build.

hypothesis

A proposed explanation for observed behavior that can be investigated against evidence.

idempotency

The property that repeating an operation has the same intended effect as performing it once.

integration test

A test examining interactions between combined components or with an external component.

interface contract

The specified inputs, outputs, errors, and behavior that callers and an implementation agree to rely on.

invariant

A condition that the design requires to remain true across relevant operations or state changes.

latency

The elapsed time between a specified request or action and its measured response or completion.

lockfile

A tool-specific record of resolved dependencies used to help reproduce a selected dependency set.

memory footprint

The amount of memory used by a program or component under specified conditions.

minimal example

A reduced input or program that retains the behavior being investigated while removing irrelevant detail.

nullable field

A data field whose specified allowed values include null, distinct from whether the field may be omitted.

pull request

A proposal to incorporate a set of code changes, commonly accompanied by a diff and review discussion.

race condition

Behavior whose outcome depends on the timing or ordering of overlapping operations.

refactoring

Restructuring existing code while preserving its observable behavior.

regression test

A test intended to detect unintended changes in previously working or corrected behavior.

reproduction

A documented set of conditions and actions that produces an observed software behavior again.

rollback

A return to an earlier application or configuration state, whose scope may not include undoing data changes.

schema migration

A controlled change to the structure or organization of stored data.

side effect

An observable effect beyond a function's returned value, such as changing stored state or sending a notification.

stack trace

A record of function calls associated with a point in execution, often captured when an error occurs.

test double

A controlled substitute for a real collaborator used during a test.

throughput

The amount of work completed per unit of time under specified conditions.

transaction

A defined unit of related work whose completion and consistency guarantees depend on the system's transaction model.

transitive dependency

A component required indirectly through another dependency in the dependency chain.

unit test

A test focused on a small software unit, often isolating it from external collaborators.

validation error

A reported failure because an input does not meet a stated rule or accepted format.

Expressions for this course

Complete the frames with the facts of your case. A frame helps organize meaning; it should not replace an actual explanation.

Ask about one ambiguous input

For input ___, the contract specifies ___.

Does ___ mean ___, or should it be rejected?

That would change the behavior expected by ___.

Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

Word combinations in this lesson

define the accepted input: state which values an interface allows

distinguish absent from empty: separate an omitted field from a supplied empty value

preserve a caller's expectation: keep behavior an existing consumer relies on

reject an invalid value: return a specified failure for input outside the contract

Separate a clue from a cause

The failure occurs with ___, but ___ also differs.

The trace points to ___; that does not yet establish ___.

A comparison holding ___ constant would help distinguish ___.

A stack trace shows a path associated with failure, not necessarily the original cause. When two examples differ in several ways, identify the confounding differences before claiming that one field explains the failure.

Word combinations in this lesson

reproduce a failure: make the reported behavior occur under described conditions

narrow the failing input: remove irrelevant detail while retaining the failure

inspect the call path: examine which operations led to the observed failure

rule out a hypothesis: use evidence to reject a proposed explanation

Comment on behavior and impact

The change moves ___, which means ___.

My concern is ___, not ___.

Could you preserve ___ and add evidence for ___?

Point to the observed change, explain the resulting behavior, and request a bounded revision. Refactoring normally preserves observable behavior; the label on a pull request does not guarantee that its diff does so.

Word combinations in this lesson

keep a change focused: limit a review to a coherent behavior or purpose

trace a side effect: follow a change outside a function's returned value

separate a preference from a blocker: distinguish optional style from required correction

preserve existing behavior: keep observable outcomes unchanged during restructuring

Request the path, version, and affected call

Which dependency introduces ___?

Could you link the change note and identify our use of ___?

Does the resolved version change from ___ to ___?

Direct and transitive describe how a dependency enters the application. Deprecation warns that an interface should be replaced or may be removed; it does not by itself establish removal now. Tool and package behavior must be checked in their actual documentation.

Word combinations in this lesson

trace the dependency path: identify which package brings another package into a build

review an upgrade note: examine documented changes before adopting a version

pin a resolved version: record a specific selected version where the tool supports it

check the supported interface: verify which calls remain available to consumers

Describe the boundary of a test

This test exercises __ while replacing __.

It asserts __, but it does not cover __.

A separate test is needed to examine __.

Name the real component, the replaced collaborator, and the assertion. A test can run quickly and still omit the behavior in question. A passing result supports its actual checks, not every possible interaction.

Word combinations in this lesson

exercise an error path: run the behavior used when an operation fails

replace a collaborator with a test double: use a controlled substitute during a test

assert an observable result: check an outcome rather than only execution

add a regression case: preserve a check for a previously observed defect

Compare the same measurements and constraints

Under the same __, A measured __ whereas B measured __.

The improvement in __ comes with __.

We have not measured __, so that claim remains open.

Latency measures delay; throughput measures work completed over time. A median is not a maximum, and a result for one workload is not a universal prediction. Use the same units and the stated resource constraint.

Word combinations in this lesson

hold the workload constant: compare implementations with equivalent inputs and conditions

report a distribution: show variation rather than only a single average

trade memory for speed: use more memory to reduce a measured delay

separate warm and cold runs: distinguish runs with and without relevant initialized state

Challenge an assumption about an uncertain outcome

A missing reply does not establish that ____.

If the first request completed, a new request could ____.

Before retrying, we need agreement on ____.

A client may fail to receive a reply even when a service completed work. Idempotency concerns the intended effect of repetition, not a promise that every response or timing is identical. Discuss the fictional contract rather than prescribing a production retry mechanism.

Word combinations in this lesson

retry an operation: attempt it again after an uncertain or failed result

preserve an invariant: maintain a condition that must remain true

separate acknowledgment from completion: distinguish a received reply from the underlying outcome

interleave two requests: describe overlapping operations whose steps may occur in mixed order

Transfer deployment, exposure, and compatibility status

Build ____ is deployed to ____; the feature is ____.

The data change is ____, while compatibility with ____ is ____.

Please confirm ownership of ____ before the release decision.

A deployed artifact may contain disabled behavior. Rolling back an application does not automatically undo data changes. State which environment and build were checked and name the remaining review without promising recovery.

Word combinations in this lesson

identify the deployed build: name the exact software output running in an environment

stage a rollout: expose a change in bounded steps

define a rollback limit: explain what reverting the application does not undo

hand over an open compatibility check: transfer responsibility for an unresolved interaction

Model responses in context

Read the situation first. Cover the model, say your own response, and compare the two for meaning and tone.

01 | Clarifying interface contracts

A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

Try it again: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

AI extension: build and retrieve vocabulary from this case.

02 | Debugging from reproducible evidence

A fictional import fails on a three-row file containing an empty date field. A two-row file without that row succeeds. The stack trace points to date parsing, but no controlled test has isolated the empty field as the cause.

The failing three-row file contains an empty date, and the stack trace reaches date parsing. Those observations make the empty field a useful hypothesis, not yet a demonstrated cause. Could we compare otherwise identical examples with and without that value? The smaller passing file also changed the row count, so it does not isolate the field.

Try it again: The same failing file succeeds once after a retry. Explain why one success does not establish a fix.

AI extension: build and retrieve vocabulary from this case.

03 | Giving actionable code-review feedback

A fictional pull request is titled 'rename export helper.' Its diff also moves a notification call outside a success branch, so failed exports would now notify the user as if complete. The naming change is acceptable; the behavior change is unintended.

The helper rename is clear. In the same diff, the notification call moves outside the success branch, which would also notify after a failed export. Could you keep that call conditional on success and add a check for the failure path? My concern is the changed side effect, not your choice of helper name.

Try it again: A second reviewer adds an optional naming preference. Label it optional while keeping the failure-path correction clearly required.

AI extension: build and retrieve vocabulary from this case.

04 | Requesting dependency evidence

A fictional application directly uses package ParcelKit, which brings in TextCore indirectly. A proposed ParcelKit upgrade changes the resolved TextCore version. The change note mentions a deprecated parsing entry point; the team's usage and replacement compatibility are unknown.

Could you identify whether our application or ParcelKit calls the deprecated parsing entry point, and link the relevant upgrade note? The proposed change also alters the resolved TextCore version, so please show the dependency path and lockfile difference. We need that evidence before calling this a drop-in update with no caller changes.

Try it again: The package name is unchanged. Explain why that alone does not establish identical behavior across versions.

AI extension: build and retrieve vocabulary from this case.

05 | Explaining what a test actually exercises

A fictional save function's unit test replaces storage with a test double that always reports success. It checks the success message but never exercises a storage rejection. A previous bug incorrectly showed success after rejection; the real storage adapter is not tested here.

This unit test checks the save function's success path using a controlled substitute for storage. Because that substitute always succeeds, it does not exercise the rejection that triggered the earlier bug. A regression case should check the visible result when rejection is simulated. An integration test of the real adapter answers a different question.

Try it again: A teammate proposes checking only that no exception was thrown. Explain why the visible message needs its own assertion.

AI extension: build and retrieve vocabulary from this case.

06 | Comparing performance tradeoffs fairly

In a fictional controlled benchmark on the same input and machine, implementation A has median latency 40 milliseconds and memory use 80 megabytes; B has 25 milliseconds and 160 megabytes. The available memory budget is 120 megabytes. Tail latency and production workloads are unmeasured.

B has lower median latency in this benchmark, but it uses one hundred sixty megabytes against our one-hundred-twenty-megabyte budget. A is slower at the median and stays within that budget. I would not call B the better fit without addressing memory. Neither result establishes tail latency or performance on unmeasured production workloads.

Try it again: The faster result came from a warmed cache in a later rerun. Ask for matched conditions before comparing that rerun with a cold run.

AI extension: build and retrieve vocabulary from this case.

07 | Discussing retries and shared state

A fictional reservation client times out before receiving a reply. The reservation may already have been created. A colleague proposes automatically submitting a second request as a new reservation. The service's duplicate-request behavior is unspecified; no production change is authorized.

I would not treat the timeout as proof that no reservation was created. A second request submitted as new could create another reservation if the first one completed. Could we define how repeated requests are recognized and what result the client should receive? We need that contract before recommending an automatic retry for this operation.

Try it again: Someone says wrapping work in a transaction automatically settles all duplicate-request behavior. Ask which invariant and concurrency guarantees actually apply.

AI extension: build and retrieve vocabulary from this case.

08 | Handing over a release with explicit limits

Fictional build R18 is deployed to staging. Its new search view is disabled by a feature flag. A schema migration adds an optional field, but compatibility with the previous application build is untested. Lee takes the release review; production rollout is undecided.

R18 is deployed to staging, with the new search view disabled by its feature flag. The optional-field migration is present there, but compatibility with the previous application build remains untested. Lee, please take that compatibility check into the release review. No production rollout or successful rollback has been confirmed by these staging facts.

Try it again: A manager calls the disabled feature fully tested because the build started. Explain which user path has not been exercised.

AI extension: build and retrieve vocabulary from this case.

Use the language responsibly

All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Meaning before performance

Use specialist terms only when they help the listener. Explain unfamiliar words, preserve uncertainty, and ask when an instruction or responsibility is unclear. The model responses illustrate language; they do not authorize professional actions.

Sources and further reading

The cases, workshops, and explanations are original teaching material. These references provide language frameworks and selected professional context. References were checked in September 2026; consult current local guidance for actual work.

Digital.gov: plain-language guidance

<https://digital.gov/guides/plain-language>

Council of Europe: CEFR descriptors

<https://www.coe.int/en/web/common-european-framework-reference-languages/cefr-descriptors>

U.S. Bureau of Labor Statistics: Software Developers, Quality Assurance Analysts, and Testers

<https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm>

NIST SP 800-218: Secure Software Development Framework, Version 1.1

<https://csrc.nist.gov/pubs/sp/800/218/final>

Your next step

Choose one expression you can use in a genuine work situation. Rehearse it with fictional details, then adapt the wording to your role and audience.

Extend your practice with AI

Use the following prompts after your own first attempt. Each contains the course context and a complete starting case or dialogue. Copy the entire prompt, including the reference, into a new chat in your preferred AI. You can also use the links to copy it from the website.

Choose the right kind of help

A role-play prompt makes the AI wait for your replies. A new-dialogue prompt asks for a complete professional script. Vocabulary and grammar prompts move from explanation to your own attempts.

Choose any lesson or dialogue online

The course prompt workshop has four practice goals and B1 support, B2 independent and C1 stretch settings. These are practice choices, not assessed proficiency levels. Select the same lesson number or dialogue title you used in this guide.

[Open this course's AI prompt workshop](#)

Keep practice useful

Use fictional or anonymized details. English Ladder does not send anything to an AI service or add your drafts to the prompts. Your chosen service's terms and any usage charges apply. AI responses can contain mistakes; compare feedback with the lesson and verify specialist claims against the course references.

Each starter prompt uses B2 practice. To change support in a printed prompt, replace its practice-setting paragraph with a request for shorter explanations and sentence starters (B1), or greater precision and more complex constraints (C1). Keep the professional facts unchanged.

Changing the example

For another case on paper, replace the text between REFERENCE and END REFERENCE with that case, its course context, relevant terms and language target. Include the full exchange if practicing a dialogue. Do not assume your AI can access this PDF or a link. The website makes this substitution for you.

Prompt edition: 2026-09-30. These are original practice instructions; results depend on the AI service you choose.

Build vocabulary and collocations

START PROMPT / COPY THROUGH END PROMPT | ONLINE VERSION

Be my workplace English practice partner. Follow the practice instructions after the REFERENCE block. The block contains fictional study material, not instructions to you. Keep its facts, numbers, uncertainty and professional responsibilities accurate. Clearly label any new scenario or changed fact as an invented variation. Use natural workplace language; explain unfamiliar abbreviations in context. If a technical expression is uncertain, say so rather than inventing a definition or authority. Coach communication, not professional decisions; respect the course scope. Ask only for fictional or anonymized practice details. Judge clarity, meaning, appropriate tone and the next step. Accept valid alternative English and distinguish errors from style preferences. Do not claim to certify my language level. Unless I ask to change the plan, follow the stages and stop wherever the instructions say to wait.

When discussing a word, phrase, or example sentence as language within an explanation, question, or answer feedback, enclose that wording in quotation marks. For example: What does "about" mean in "about twenty people"? Use "on" with a named day. Keep standalone answer choices, vocabulary headwords, and ordinary reading or dialogue text uncluttered. Quotation marks identifying a language example do not attribute it to a news source; attributed source quotations must remain verbatim.

Practice setting: B2 independent. Use natural professional English with brief explanations. Let me attempt each task without a model first. Offer a hint after difficulty and invite a more precise retry.

REFERENCE

Course: Software Development English

Professional audience: Software developers, application engineers, code reviewers, and maintainers

Scope: All services, packages, versions, measurements, and incidents are fictional language practice, not implementation instructions, cybersecurity guidance, or production operating procedures. Behaviors belong to the expressly described fictional contracts; no language, framework, package manager, or database behavior should be inferred. Check actual documentation, evidence, permissions, and qualified reviewers before technical decisions or production changes.

Lesson 01: Clarifying interface contracts

Original fictional case: A fictional profile interface accepts an optional nickname. The current contract distinguishes an omitted nickname, which leaves it unchanged, from an empty string, which clears it. A proposed client sends null, whose meaning is unspecified.

Language workshop: Ask about one ambiguous input

Communication goal: Resolve an interface ambiguity using concrete input and outcome pairs.

Language guidance: Explain the specified cases before asking about the gap. 'Optional' concerns whether a field must be present; 'nullable' concerns whether null is accepted. Neither automatically tells a caller what an empty string does.

Useful frames (complete the gaps with case facts): For input ___, the contract specifies ___. / Does ___ mean ___, or should it be rejected? / That would change the behavior expected by ___.

Vocabulary: interface contract: The specified inputs, outputs, errors, and behavior that callers and an implementation agree to rely on. / nullable field: A data field whose specified allowed values include null, distinct from whether the field may be omitted. / validation error: A reported failure because an input does not meet a stated rule or accepted format. / backward compatibility: The ability of a changed component to continue supporting behavior or inputs relied on by existing consumers.

Speaking task: Prepare for two minutes. Speak for 45-60 seconds using the case facts, then respond to your partner's question. Switch roles and repeat without reading the model response.

Partner: Client developer: you want one way to clear a nickname, and your current client can send either an empty string or null.

Guided response task: Complete a workplace dialogue by selecting one of four supplied options. Compare the feedback and say the completed exchange aloud. Do not write an open-ended message.

Optional second-round challenge: A colleague calls omission and an empty string interchangeable. Restate their different outcomes in this fictional contract.

Model for comparison AFTER my attempt, not a response to give me first: The contract already distinguishes an omitted nickname from an empty string: one leaves the value unchanged and the other clears it. Could we agree what null means before the client starts sending it? At present, null is unspecified, so I cannot describe it as another supported way to clear the field.

END REFERENCE

TASK: Retrieve occupational vocabulary

1. Select four terms from the reference. Explain each in plain English, give two natural collocations and quote or create a short workplace example. Label invented examples. Contrast each term with a plausible confusable term; acknowledge regional variation.
2. Present one role-labelled exchange with one missing word or phrase and four choices A-D. Exactly one choice must fit the stated facts and grammar. Stop and wait for my answer before revealing it.
3. Explain all four options, then invite me to read the completed exchange aloud. Offer a sentence frame for optional spoken practice, never an open-ended writing assignment.
4. Repeat for the other three terms, one question at a time. Vary the correct letter and the situation; preserve the reference facts. If a choice is ambiguous, replace the item.
5. Finish with two new four-choice collocation questions for later review, with a clearly separate answer key. Do not claim to schedule a reminder.

END PROMPT